problem

This is **NOT runnable code**.

Missing things like:

- `#include <string.h>`
- maybe struct definitions
- maybe other functions

So compiler says: error: strcpy not defined

simple attempt

function + main() → make it runnable

Example:

```
int main() {
   func("test");
}
```

Why this FAILED

Because:

- function depends on things **not present**
- dataset removed context

So: compile fails

Context = everything around the function

Includes:

- headers (`#include`)
- structs
- macros
- other functions
- libraries

Example:

```
struct Data {
   int x;
};
```

If missing → code breaks.

reconstruct original context

Step 1: Find where function came from

Dataset gives:

commit_id
project_name

This tells: "Which GitHub project and which version"

Step 2: Clone project

download full codebase from GitHub

Now we have: real project files

Step 3: Find the function

Search inside project files: original file with full code

Step 4: Extract headers

From that file: #include <stdio.h>, #include <string.h>

What is harness.c?

Still, we need to **run the function automatically**

So created:

```
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
   func(data);
   return 0;
}
```

“Take input and pass it to function repeatedly”

Fuzzing needs: automatic repeated execution

Input generation

We generate different types of inputs to test the function under multiple conditions.

3 types of inputs

1. Structured inputs

JSON, XML, HTTP-like inputs
“These simulate real-world formatted inputs.”

2. Edge-case inputs

empty string, very long strings, null bytes
“These are designed to trigger boundary conditions like buffer overflow.”

3. Random inputs

random characters

“These explore unexpected execution paths.”

For each run, it randomly chooses one type of input and feeds it to the program.

This is called fuzzing — repeatedly testing with different inputs.

Output

First sanitizer features

All zeros:

heap_overflow = 0
use_after_free = 0
No memory or undefined behavior errors were triggered during execution.
Because vulnerabilities require specific input conditions, and not all inputs trigger them.

Behavioral features

timeouts = 0

> “The program did not hang or enter infinite loops.”

avg_exec_time = 0.0239

> “Execution time is small, so the function runs normally.”

unique_exit_codes = 1

> “The program follows the same execution path every time.”

stderr_len = 91.68

> “Some output is generated, but no error messages.”

success_rate = 1.0

> “All executions completed successfully.”

“Dynamic analysis depends on actually triggering vulnerabilities, but our dataset contains isolated functions without full context, so triggering those conditions is difficult.”

“Even with fuzzing, random inputs may not satisfy the exact conditions needed to expose a bug.”

“To overcome this, we didn’t rely only on crashes. We also captured execution behavior such as timing, stability, and output patterns.”

“Even when vulnerabilities are not triggered, these behavioral features vary across functions and provide useful signals for the model.”

“Dynamic analysis is difficult to scale because it requires actual execution of each program multiple times with different inputs.”

1. Core reason (most important)

Example:

For **1 file**:

50 runs per file

For **1000 files**:

1000 × 50 = 50,000 executions

For **10,000 files**:

10,000 × 50 = 500,000 executions

Huge increase

2. Time complexity problem

Each run takes time:

execution + input generation + logging

Even if 1 run = 0.02 sec:

- 50,000 runs → ~1000 sec (~17 min)
- 500,000 runs → ~10,000 sec (~3 hours)

3. Compilation difficulty (BIG issue)

Before execution:

- each function must be compiled
- needs headers, dependencies

Problem:

Dataset functions are incomplete

So:

- many fail to compile
- need reconstruction (very heavy work)

4. Input generation challenge

Dynamic analysis depends on:

correct input → triggers vulnerability

Problem:

- we don’t know correct input
- fuzzing is random
- many runs give no result

5. Low bug trigger probability

Even after many runs:

no crash, no sanitizer trigger

Why?

- wrong inputs
- safe execution path
- missing context

6. Resource usage

Dynamic analysis needs:

- CPU (many executions)
- memory (process spawning)
- time (large loops)

7. Output processing overhead

For each run:

- capture stderr
- analyze logs
- update features

Adds more cost

8. Why 1k is already difficult

Even 1k:

- thousands of executions
- compilation issues
- debugging time
- inconsistent results

9. Why 10k becomes VERY hard

Because everything scales:

Factor	1k	10k
executions	50k	500k

time	medium	very high
failures	manageable	huge
debugging	possible	difficult

“Dynamic analysis is computationally expensive because it requires repeated execution of each program with multiple inputs. As the dataset size increases from 1k to 10k, the number of executions grows significantly, making it time-consuming and resource-intensive. Additionally, incomplete code context and input dependency further reduce the effectiveness of dynamic analysis at scale.”

TYPES OF FUZZING WE ADD

1. Overflow fuzzing

very large inputs → trigger buffer overflow

2. Null/invalid input fuzzing

NULL bytes, empty → null dereference

3. Numeric fuzzing

very large numbers → integer overflow

4. Format fuzzing

%x %s → format string bugs

5. Mutation fuzzing

modify previous inputs → explore paths

BEST SIMPLE TECHNIQUES

1. PATH DIVERSITY (VERY POWERFUL)

Idea:

Track how differently program behaves for different inputs

Final feature:

"path_variability": len(path_signatures)

Why it helps:

Even if no crash: different behavior → different paths

gives strong signal

2. OUTPUT DIVERGENCE (VERY IMPORTANT)

Idea:

Measure how much output changes across runs

Final:

"unique_outputs": len(set(stderr_outputs))

Why:

Different outputs = different execution logic

3. EXECUTION VARIANCE (VERY IMPORTANT)

Final:

import numpy as np
"time_variance": float(np.var(times))

Why:

unstable execution = suspicious behavior

4. INPUT SENSITIVITY (VERY POWERFUL)

Idea:

Does output change when input changes?

Final:

"input_sensitivity": input_effect

5. EXIT BEHAVIOR COMPLEXITY

You already have: unique_exit_codes

Keep it — very useful

6. STDERR INTENSITY

Instead of avg only:

Add:

```
max_stderr = max(len(result.stderr), r.get("max_stderr", 0))
r["max_stderr"] = max_stderr
```

Final:

```
"max_stderr_len": r["max_stderr"]
```

7. TIMEOUT BEHAVIOR

Already have: timeouts

keep — very strong signal

FINAL FEATURE VECTOR (UPGRADED)

Now your vector becomes:

crash_count
crash_rate

asan / ubsan (still there)

timeouts
avg_exec_time
time_variance

unique_exit_codes
stderr_len
max_stderr_len
unique_outputs

path_variability
input_sensitivity

Success_rate

Sample Context

- No crashes
- High variability
- Same exit code

So this is a **stable but behaviorally complex binary**

1. `crash_count = 0`

➢ What it measures

Number of runs where:

- non-zero exit OR
- “error” in stderr

➢ What it means

No obvious crash triggered

➢ What it captures

- hard failures
- visible bugs

➢ Example

Input `"A"*50000` → still runs fine
👉 no segfault → stays 0

2. `crash_rate = 0.0`

➢ What it measures

crash_count / total_runs

➢ What it means

0% failure rate → stable execution

➢ Captures

Reliability under fuzzing

3–7. Memory bug indicators (all 0)

- `heap_overflow_count`
- `stack_overflow_count`
- `use_after_free_count`
- `double_free_count`
- `invalid_access_count`

➢ What they measure

Detection via stderr patterns (ASAN-like)

➢ What it means

> No memory safety violations triggered

➢ What it captures

Classic vulnerabilities

➢ Example

If program had:

char buf[10];

gets(buf);

large input → stack overflow → count increases

8–11. Undefined behavior indicators (all 0)

- `int_overflow_count`
- `divide_by_zero_count`
- `null_deref_count`
- `invalid_shift_count`

➢ What they measure

UBSAN-type issues

➢ What it means

> No undefined behavior observed

12. `timeouts = 0`

➢ What it measures

Executions exceeding TIMEOUT

➢ What it means

> No hangs / infinite loops

➢ Example

If input `"AAAA..."` causes loop → timeout++

13. `avg_exec_time = 0.07692`

➢ What it measures

Average runtime per input

➢ What it means

> Moderately slow execution (not trivial)

➢ What it captures

- algorithm complexity
- input processing cost

➢ Example

- `"A"` → fast

- `"A"*50000` → slower
  👉 increases avg time

14. `unique_exit_codes = 1`

➤ What it measures

Number of distinct return codes

➤ What it means

Program always exits the same way (likely `return 0`)

➤ What it captures

Final state diversity (coarse)

➤ Example

All inputs → exit code 0

15. `stderr_len = 183.16`

➤ What it measures

Average stderr length

➤ What it means

Program consistently produces output/errors

➤ What it captures

- logging
- validation messages

➤ Example

Each input prints: Invalid input: X

16. `success_rate = 1.0`

➤ What it measures

% of runs with exit code 0

➤ What it means

Fully stable execution

17. `path_variability = 50`

➤ What it measures

Unique hashes of:

(stderr + returncode)

➤ What it means

Every run behaved differently

➤ What it captures

execution path diversity (approximation)

➤ Example

Input	Output
"A"	error 1
"B"	error 2

"C" error 3

👉 all different → high variability

18. `unique_outputs = 50`

➤ What it measures

Distinct stderr outputs

➤ What it means

Every input produced different output

➤ What it captures

Output diversity → logic diversity

19. `time_variance = 0.032`

➤ What it measures

Variance of execution time, high variance

➤ What it means

Runtime strongly depends on input

➤ What it captures

- branching complexity
- loops / heavy computation

➤ Example

Input	Time

small 0.01s

large 0.12s

20. `input_sensitivity = 49`

➤ What it measures

Number of times output changed between consecutive inputs

➤ What it means

Almost every input changes behavior

➤ What it captures

Sensitivity of logic to input changes

➤ Example

"A" → output1, "B" → output2, "C" → output3

always different → high sensitivity

21. `max_stderr_len = 184`

➤ What it measures

Maximum stderr length seen

➤ What it means

Some inputs trigger slightly larger outputs

➤ What it captures: Worst-case output size / verbosity